

Rainbow Source Library: 会议纪要与计算架构 Tutorial

这份 report 把一次关于 Rainbow 信源库的会议整理成可执行的架构教程。它不是逐字会议记录，而是把会议里出现的产品判断、代码结构、计算链路、短期 TODO 和长期设计取舍放到同一个解释框架里。

读法很简单：先看 TLDR，再按图理解每个模块为什么存在，最后看 TODO 和开放问题。

TLDR

信源库是整个平台的供给侧。它不只是给前端展示分类，也是在给 Agent 提供低成本、高质量、可追溯的上下文。

会议形成了两个并行工作面：

- 信源治理：质量、分类、标签、准入准出、垂类推荐源。
- 信源能力：时效性、SQL/source-store 集成、检索、评测、Agent 消费链路。

短期最大瓶颈是 source-store 维护。当前信源覆盖、时效、多样性偏弱，而且更新偏手动/异步。Qihang 提出的 P0 是把 `zsearch` 输出接进 SQL/source library，让新 URL 可以进入现有检索链路。

周五目标更像一个 investor-agent pressure test：能不能从投资人相关信源里捞数据、定位信息、生成回答，并且能追溯来源。

1. 会议到底在对齐什么

会议一开始的分歧不是某个字段怎么命名，而是谁在做哪一层、信源这件事到底属于产品、数据、检索还是 Agent。

最后可以归纳成一句话：

信源不是一个静态列表，而是一套从外部信息供给到 Agent 交付结果的计算系统。

会议里涉及的工作包括：

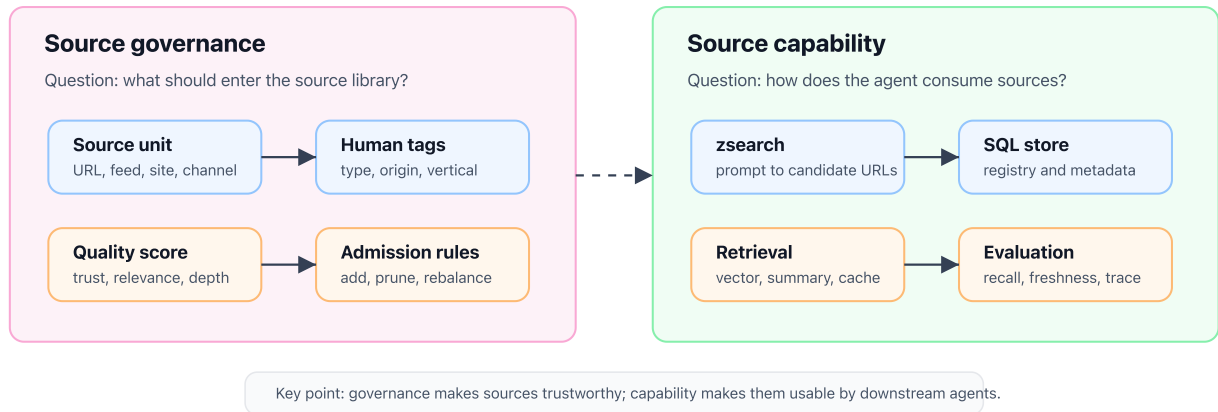
- 在新标签体系下整理存量外部信源。
- 建立质量评分、准入准出、更新和清理机制。
- 支持未来用户自己的信源。
- 让信源元数据同时服务用户前端和 Agent 检索。
- 把信源维护和下游任务型产品连接起来。

2. 第一张图：信源工作要拆成两条线

这张图解释会议里反复出现的一个关键分工：信源治理和信源能力不是同一件事。

Meeting Split: Source Governance vs Source Capability

The meeting becomes clearer once source work is split into policy/quality work and runtime/computation work.



Source work split

会议纪要

Xu Hao 把问题拆成两块：一块是信源库本身的管理，另一块是信源能力。前者关心这个源是否存在、质量如何、应该打什么标签；后者关心它如何被实时更新、如何被检索、如何给 Agent 提供能力出口。

Murphy 也把信源标签理解成两层：一层给用户展示，帮助筛选和建立信任；另一层给检索和 Agent 使用，帮助后续任务找到对的材料。

Qihang 的补充是：对 Agent 来说，标签未必是最有用的结构。更有用的可能是 summary、cluster、topic cache 这种更接近语义地图的中间层。

Tutorial 解释

信源治理回答的是：“什么信息值得进入系统？”

信源能力回答的是：“这些信息如何被计算系统及时、低成本、可评测地消费？”

如果只做治理，前端看起来会更整齐，但 Agent 的回答未必更好。如果只做能力，系统可能搜到很多东西，但没有垂类边界、质量判断和来源信任，结果会不稳定。

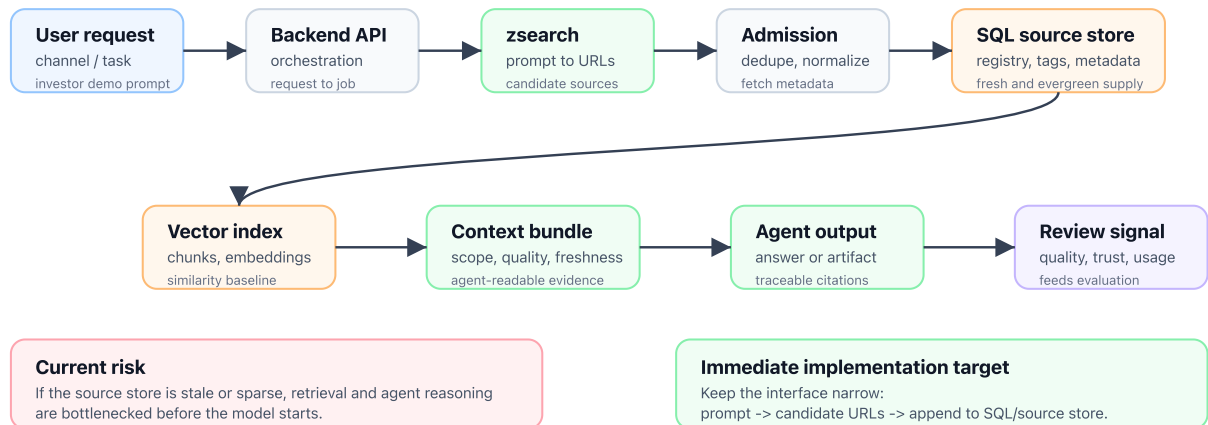
所以这两个方向要并行推进。

3. 第二张图：从用户请求到 Agent 输出的代码结构

这张图把会议里提到的代码/系统链路压缩成一个 hot path。它描述的是用户发起一个 channel/task/investor demo 请求之后，系统大致需要经过哪些层。

Near-term Hot Path: Make zsearch Feed the Source Store

This is the narrow implementation cut discussed in the meeting: do not rebuild everything first, connect prompt-to-URL discovery into the existing retrieval substrate



Runtime hot path

会议纪要

Qihang 对现有链路的理解是：

1. 用户发出 request。
2. 系统 search 相关 material。
3. 新 URL 或 source material 被异步加入 SQL。
4. Murphy 负责信源打标或治理链路。
5. Evan 侧的检索能力从 SQL/vector path 里找材料。
6. Agent 基于检索结果生成用户看到的 channel/task response。

Qihang 提出的短期接口可以先保持极窄：

```
input: prompt
output: list of URLs
side effect: append usable URLs into SQL/source store
```

Tutorial 解释

这个链路里最重要的判断是：`zsearch` 不应该只是一个一次性的搜索工具，而应该变成 source-store 的补给入口。

短期不需要重写整个后端。更实际的做法是把接口先固定住：

```
prompt -> zsearch -> candidate URLs -> admission glue -> SQL/source store
```

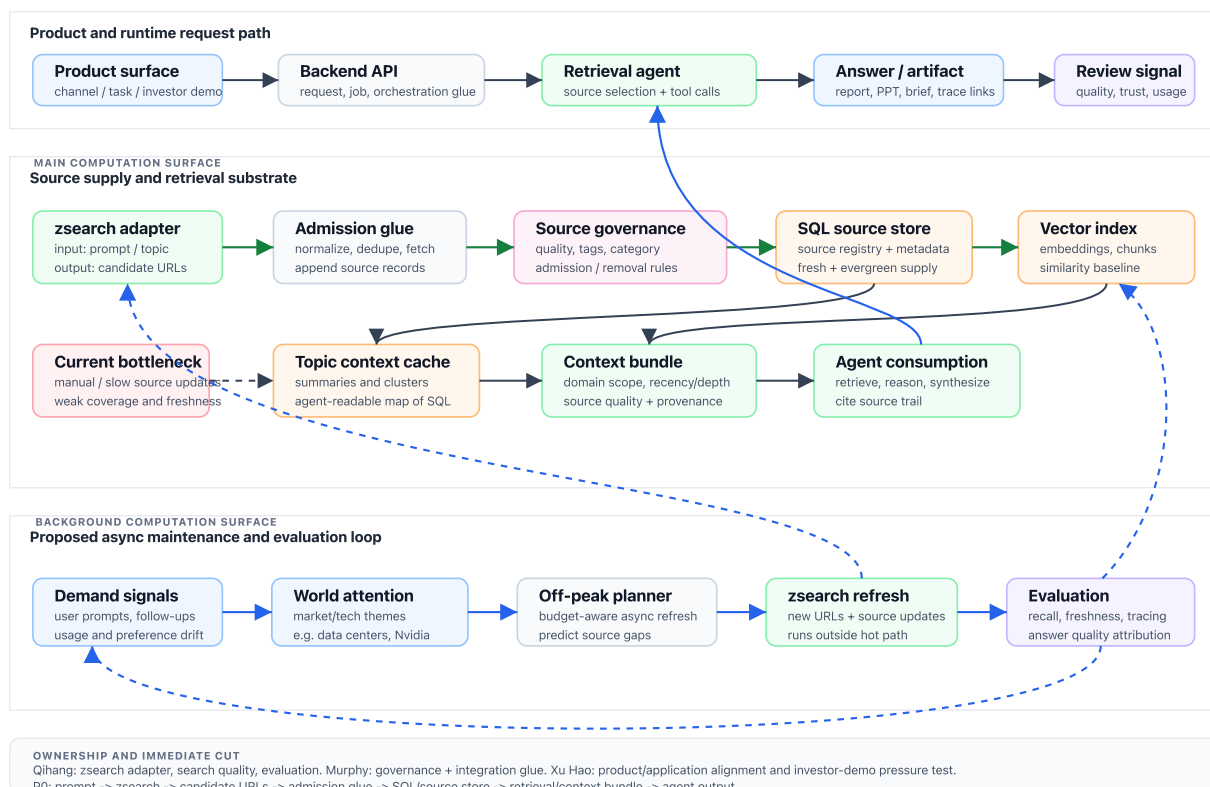
只要 URL 能稳定进入 source store，后面的 vector index、context bundle、Agent consumption 就可以沿用现有路径做 pressure test。

4. 第三张图：为什么 source-store 是当前瓶颈

会议里最清楚的短期判断是：系统现在最大的功能性瓶颈不是 Agent 不能写，而是 source store 的供给不够好。

Rainbow Source Library: Code Structure + Proposed Computation Architecture

Goal: make source supply reliable enough for investor/task agents while keeping runtime cost and evaluation complexity under control.



Full architecture overview

会议纪要

Qihang 的判断是，当前 source library 里的 URL 数量、时效性和 diversity 都不够。现有维护方式偏手动，可能只是某个人用 skill 固定更新一下。这样会让整个系统的表现受限，因为检索只能在已有 SQL/source store 里找。

Murphy 观察到当前源库里 podcast 或 call 类型占比很高，这说明存量源可能需要重新平衡和治理。

Xu Hao 关心的是产品落地：对投资人、程序员等不同垂类，系统需要有更可靠的内置源和更可控的检索入口，否则任务输出很难稳定。

Tutorial 解释

这其实是一个供给侧问题：

- 如果 SQL/source store 里没有足够多、足够新、足够垂类的材料，Agent 再强也只能在贫弱上下文里推理。
- 如果 source store 更新慢，用户看到的 channel 可能还是几天前的内容。
- 如果 source store 没有质量和垂类边界，vector search 可能召回跨领域、低质量或不够可信的材料。

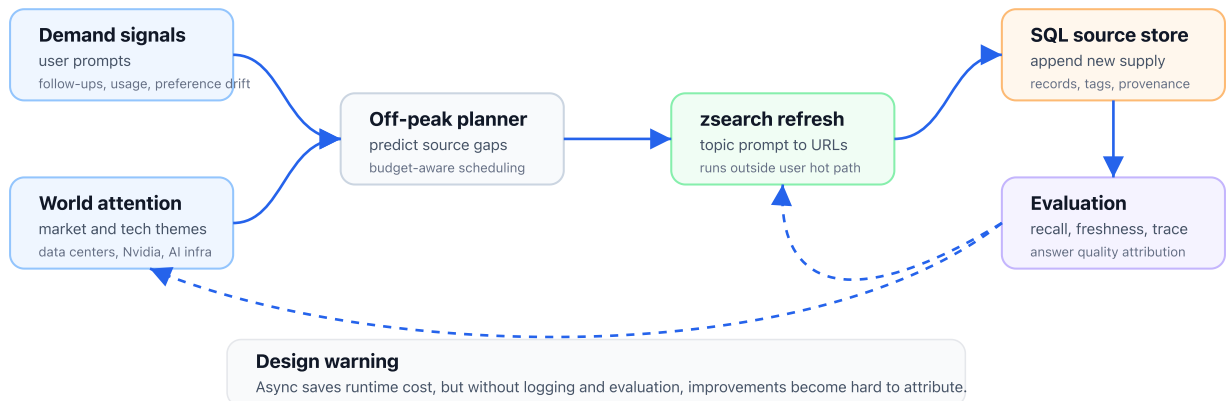
所以 P0 不应该先变成复杂 Agent workflow，而是先把供给侧补起来。

5. 第四张图：异步维护和评测闭环

会议里还有一个更长期的设计：不是每一次都等用户来了才搜索，也可以在低峰期根据用户需求和世界主线异步维护 source store。

Background Loop: Async Refresh Needs Evaluation

The proposed architecture can refresh sources off the hot path, but every refresh should leave enough traces for attribution.



Async maintenance loop

会议纪要

Qihang 提到，可以根据用户请求分布和世界 attention 主题做预测式维护。例如用户群体近期都在问数据中心、Nvidia、AI infra，那系统可以在 API 成本更低的时候异步刷新这些主题的源。

但 Qihang 同时强调，异步设计会让 evaluation 变难。因为 source store 是累积变化的，当最终答案变好或变差时，很难判断是因为新 URL、标签、检索、vector index，还是 Agent reasoning。

Tutorial 解释

异步维护的好处是：

- 避免每个用户请求都阻塞在新搜索上。
- 可以利用低峰期做 cheaper computation。
- 可以让 source store 的分布逐渐贴近用户需求分布。

异步维护的代价是：

- 更难归因。
- 更难知道哪个 source update 带来了结果提升。
- 更需要 evaluation logging 和版本化。

所以 background refresh 不能只做 append。它应该同时记录：

- 为什么刷新这个 topic；
- `zsearch` 用了什么 prompt；
- 加入了哪些 URL；
- 进入 SQL/source store 后对 recall、freshness、answer quality 有什么影响。

6. 标签、summary、cluster 各自解决什么问题

会议里一个容易混淆的点是：标签是不是给 Agent 用的？

答案是：部分是，但不够。

用户需要标签，因为标签可以解释来源、帮助筛选、建立信任。例如：

- source type: news、podcast、report、blog、technical docs；
- origin: OpenAI、Bloomberg、company IR、Substack；
- vertical: investment、programming、AI infra。

Agent 需要的不只是标签。Agent 更需要知道：

- 这个源大概讲什么；
- 哪些 topic 和这个源强相关；
- 这个源是 fresh news 还是 evergreen depth；
- 这个源在某个垂类里可信度如何；
- 这个源应该在什么任务里被优先召回。

所以更合理的结构是：

```
human-facing tags
```

```
-> filtering, trust, UI display
```

```
agent-facing source context
```

```
-> summaries, clusters, quality, recency/depth preference, provenance
```

7. 时效性不是“越新越好”

会议里 Qihang 和 Xu Hao 对“新旧信源”的分歧最后变成了一个更清楚的原则：时效性是一种能力，不是删除旧信息的规则。

不同 source type 对时效性的偏好不同：

- 新闻、市场事件、财报追踪：强 recency bias。
- podcast、书、深度报告：更重视深度和 evergreen value。
- 投资任务：既需要新数据，也需要长期结构性理解。

所以 source store 不应该只保留新信息。它应该保留足够丰富的 source universe，然后在 retrieval/context bundle 阶段根据任务类型调权。

8. 从 channel 到 task：产品方向的变化

Xu Hao 提到，未来 channel 可能会变成 task。这个变化很重要，因为它改变了系统应该优化的对象。

Channel 更像持续刷新的信息流。Task 更像一个有目标、有步骤、有产物的工作流。

对于复杂 task，Qihang 的判断是：不能期待 Agent 立刻给出高质量结论。尤其是投资或研究任务，如果想得到 nontrivial conclusion，它天然需要后台执行、拆 subtask、查证、合成和通知。

这意味着长期架构应该支持：

- task decomposition；
- sequential / parallel subtask dependency；
- long-running background execution；
- evidence trace；
- user review and follow-up；
- preference memory。

9. TODO List

Qihang

- 把 `zsearch` 输出接进 SQL/source library path。
- 将接口先定义为 `prompt -> list of URLs -> append usable URLs into SQL/source store`。
- 重点负责 Agent search quality、evaluation，以及 source maintenance 对 retrieval performance 的影响。
- 避免被过多 generic backend glue 吸走，除非接口本身需要。
- 开 branch 后让 Murphy review integration direction。

Murphy

- 帮 Qihang 把 `zsearch` interface 串进 backend/source-library chain。
- 继续做 source governance: tags、categories、source quality、存量 pruning/balancing。
- 和 Qihang 一起设计 recall/retrieval evaluation。
- 讨论长期 search/retrieval abstraction，因为 search tool 越来越是 agent-facing，而不只是传统召回。

Xu Hao

- 负责 product/application-side alignment 和 user-facing direction。
- 明确 source governance 和 source capability 怎么支撑产品目标。
- 用 investor-agent pressure test 倒推短期优先级。

10. Open Questions

- Source tagging 应该在哪里停止，retrieval-side summarization/clustering 应该在哪里开始？
- Fresh source search 是否应该在某些 user task 中同步阻塞，还是始终异步刷新？
- Evaluation 如何把 final answer quality 归因到 source coverage、source tags、retrieval、vector index 或 Agent reasoning？
- Friday investor-agent pressure test 所需的 minimal investor source set 是什么？
- User preference 应该用 freeform Markdown、structured schema、hook/if-else rules，还是 graph-like preference store？
- 如果 channel 变成 task，当前 channel architecture 哪些部分还能复用？

11. 核心设计原则

Source work 和 Agent work 是互补关系。Storage-time work 做得越多，runtime agent 成本越低；runtime agent 越强，也可以补 storage 的不足，但成本和稳定性会变差。

异步 source maintenance 可以省成本，但会让 evaluation 变难。系统应该把 source update、retrieval 和 final-answer quality 当成一个 joint system 设计。

Tags 至少有两个受众。Human-facing tags 服务信任和筛选；agent-facing metadata 更应该包含 summary、cluster、source quality、domain scope、recency/depth preference。

Freshness 应该 source-type aware。Recency 对新闻和市场事件很重要，但不能替代 source quality 和 evergreen depth。

复杂 Agent task 应该是 graph-shaped。投资和研究工作流需要拆解、依赖、后台执行、证据追溯和 review surface。

Retrieval Keywords

Rainbow source library, ZeepBot source system, source governance, source capability, source tags, source quality, source admission, source removal, SQL source store, zsearch integration, agent retrieval, vector search, source freshness, async source maintenance, investor agent demo, channel to task, task graph, user preference learning, source supply side, meeting report tutorial.